

TRABAJO PRÁCTICO INTEGRADOR

Food Store

Grupo 56

Santino Fiorentini (Comisión 4) | David Kenny (Comisión 7)

Universidad Tecnológica Nacional
Tecnatura Universitaria en Programación
PROGRAMACIÓN II

Video explicativo: [YouTube](#)

Repositorio: [GitHub](#)

22 de junio de 2026

Tabla de contenidos

Introducción	3
Marco teórico	3
Decisiones Técnicas y Arquitectura	4
Dificultades y Soluciones	6
Conclusión	7
Bibliografía	7

Introducción:

Este trabajo consiste en el desarrollo de una aplicación de consola denominada Food Store, orientada a la gestión de categorías, productos, usuarios y pedidos.

El sistema permite realizar operaciones CRUD (crear, listar, modificar y eliminar) utilizando Programación Orientada a Objetos, colecciones y manejo de excepciones. Además, incorpora validaciones para garantizar el correcto funcionamiento de las operaciones y mantener la consistencia de los datos almacenados en memoria.

Marco teórico:

Java es un lenguaje de programación orientado a objetos ampliamente utilizado para el desarrollo de aplicaciones de distintos tipos. Una de sus principales ventajas es que permite crear programas organizados, reutilizables y fáciles de mantener.

La Programación Orientada a Objetos (POO) es un paradigma que organiza el software mediante clases y objetos.

Los conceptos aplicados fueron:

- Encapsulamiento:

Cada entidad mantiene sus atributos privados y proporciona acceso controlado mediante getters y setters.

- Herencia:

Todas las entidades heredan de una clase abstracta Base que centraliza atributos comunes como: id, eliminado y createdAt.

- Polimorfismo:

Se implementó mediante la sobreescritura de métodos como toString() y el uso de interfaces.

- Interfaces:

Se utilizó la interfaz Calculable para definir el comportamiento común de cálculo de totales en los pedidos.

- Colecciones:

La información del sistema se almacena en memoria utilizando colecciones dinámicas de Java.

Principalmente se utilizó la colección ArrayList para almacenar categorías, productos, usuarios y pedidos.

- Manejo de excepciones:

Se implementaron excepciones para controlar situaciones anómalas como: Entidad inexistente, Mail duplicado, Categoría duplicada, Stock inválido y Precio inválido.

Esto permite mantener la estabilidad del sistema y evitar interrupciones inesperadas.

- Bases de datos relacionales:

Aunque el sistema almacena la información en memoria, el modelo utilizado sigue principios similares a los empleados en bases de datos relacionales.

Se aplicaron conceptos como entidades, relaciones uno a muchos, integridad referencial e identificadores únicos.

Decisiones Técnicas y Arquitectura:

- Decisiones Técnicas:

Durante el desarrollo se decidió organizar el proyecto utilizando paquetes para separar claramente las responsabilidades de cada componente.

Las entidades se ubicaron en un paquete específico encargado de representar el modelo del negocio. Las reglas de negocio se concentraron en los servicios, mientras que la interacción con el usuario se desarrolló mediante clases de menú independientes.

También se decidió implementar bajas lógicas en lugar de eliminar físicamente los objetos de las colecciones. De esta manera se conserva la información histórica y se cumple con los requisitos establecidos en la consigna.

Otra decisión importante fue implementar excepciones personalizadas para representar errores propios del sistema, como correos duplicados, entidades inexistentes o valores inválidos. Esto permitió mantener el código más ordenado y facilitar la detección de problemas durante las pruebas.

Por último, se eligió trabajar con ArrayList para almacenar los datos en memoria, ya que la consigna no requería persistencia en bases de datos.

- Arquitectura del Proyecto:

El proyecto fue organizado en distintos paquetes con el objetivo de separar responsabilidades y mantener una estructura clara y mantenible.

El paquete entities contiene las entidades principales del sistema definidas en el UML: Base, Categoría, Producto, Usuario, Pedido y DetallePedido. Estas clases representan los objetos del dominio y almacenan tanto sus atributos como sus comportamientos principales.

El paquete enums agrupa las enumeraciones utilizadas por la aplicación: Rol, Estado y FormaPago. Su función es restringir determinados atributos a valores predefinidos y mejorar la legibilidad del código.

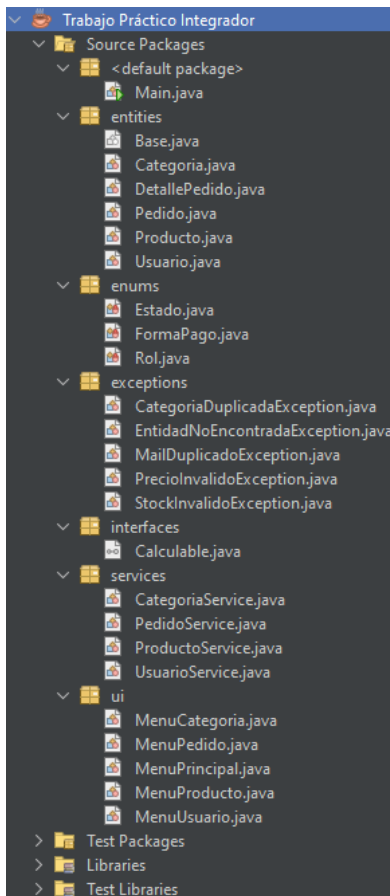
El paquete exceptions incluye las excepciones personalizadas desarrolladas para validar las reglas de negocio del sistema: CategoríaDuplicadaException, EntidadNoEncontradaException, MailDuplicadoException, PrecioInvalidoException, StockInvalidoException.

El paquete interfaces contiene la interfaz Calculable, utilizada para definir el comportamiento de cálculo de totales dentro de los pedidos.

El paquete services implementa la lógica de negocio de la aplicación mediante las siguientes clases: CategoríaService, ProductoService, UsuarioService y PedidoService. Estas clases gestionan las operaciones CRUD, validaciones y reglas del dominio.

El paquete ui contiene las clases responsables de la interacción con el usuario mediante menús de consola: MenuPrincipal, MenuCategoria, MenuProducto, MenuUsuario y MenuPedido.

La clase Main constituye el punto de entrada de la aplicación. Desde ella se inicializa el sistema y se ejecuta el menú principal que permite acceder a las distintas funcionalidades.



Dificultades y Soluciones:

Una de las primeras dificultades encontradas fue la correcta interpretación del UML proporcionado en la consigna. Fue necesario analizar cuidadosamente las relaciones entre entidades para implementar correctamente las asociaciones entre productos, categorías, usuarios y pedidos.

Otra dificultad importante fue el cálculo automático de subtotales y totales dentro de los pedidos. Inicialmente algunos valores debían actualizarse manualmente, lo que generaba inconsistencias. La solución fue implementar métodos específicos encargados de recalcular automáticamente estos importes cada vez que se modificaban los datos relacionados.

También surgieron inconvenientes relacionados con la generación de identificadores únicos para las distintas entidades. Para resolver este problema se utilizaron contadores internos que permiten asignar IDs de manera automática durante la ejecución.

El manejo de errores fue otro aspecto que requirió atención. Durante las pruebas aparecieron situaciones como correos duplicados, productos inexistentes o valores negativos. Para resolverlo se incorporaron validaciones y excepciones personalizadas que permiten informar claramente cada problema.

Finalmente, la implementación de las bajas lógicas representó un desafío adicional. En lugar de eliminar elementos de las colecciones se decidió utilizar un atributo que indica si un registro se encuentra eliminado, permitiendo conservar la información histórica y respetar los requisitos del proyecto.

Conclusión:

La realización de este Trabajo Práctico Integrador permitió aplicar los principales conceptos vistos en la materia Programación 2, como Programación Orientada a Objetos, herencia, interfaces, colecciones y manejo de excepciones.

A través del desarrollo del sistema Food Store se logró construir una aplicación funcional para la gestión de categorías, productos, usuarios y pedidos, incorporando validaciones y una organización adecuada del código.

Bibliografía:

- Material de Cátedra de Programación 2.